



UNIVERSIDAD DE CARABOBO
FACULTAD DE CIENCIAS Y TECNOLOGÍA
ESCUELA DE COMPUTACIÓN
CAMPUS BÁRBULA
ARQUITECTURA DEL COMPUTADOR

Práctica de Laboratorio 2:
Algoritmos de Ordenamiento y su Interacción con la
Memoria en MIPS32

Daniel Gutierrez CI: 32047059
Gabriel Diaz CI: 31161550

Bárbula, Agosto de 2026

1. Diferencias entre Registros Temporales (\$t0–\$t9) y Guardados (\$s0–\$s7) y su Aplicación Práctica

La diferencia fundamental entre los registros temporales (\$t0–\$t9) y los registros de guardado (\$s0–\$s7) radica en la convención de llamadas de las funciones en MIPS y en cuál rutina es responsable de salvar su información en la pila.

1.1 Registros Temporales (\$t0–\$t9)

Caller-saved (salvados por el llamador).

¿Se preservan en un jal?: NO. La función llamada puede sobrescribirlos libremente.

Responsable de la pila: La función que hace la llamada (caller). Si la función principal necesita conservar el valor de un registro \$t después de ejecutar jal, ella misma debe guardarlo en la pila antes de la llamada y restaurarlo después.

Uso principal: Variables locales de vida corta, contadores de bucles locales y cálculos intermedios de direcciones.

1.2 Registros Guardados (\$s0–\$s7)

Callee-saved (salvados por el llamado).

¿Se preservan en un jal?: SÍ. Su valor debe mantenerse idéntico antes y después de cualquier llamada a subrutina.

Responsable de la pila: La función que es llamada (callee). Si una función llamada necesita utilizar un registro \$s, está obligada a respaldar su valor original en la pila durante el prólogo y restaurarlo antes de retornar con jr \$ra.

Uso principal: Variables de vida larga y parámetros que deben sobrevivir a múltiples llamadas a subrutinas.

1.3 Aplicación en la Práctica (Caso QuickSort)

Para entender cómo se aplican, si tenemos una función Principal que llama mediante jal a una función Auxiliar:

Uso de Registros \$t: La función Auxiliar puede usar cualquier registro \$t libremente. Si la función Principal no quiere perder el dato guardado en \$t0, ella misma hace un sw \$t0, 0(\$sp) antes del jal y un lw \$t0, 0(\$sp) después.

Uso de Registros \$s: La función Principal confía en que \$s0–\$s7 no van a cambiar tras un jal. Si Auxiliar necesita usar \$s0, respalda el \$s0 original en la pila al entrar y lo restaura antes de salir.

2. Diferencias entre los Registros \$a0–\$a3, \$v0–\$v1, \$ra y su Aplicación Práctica

Estos registros definen el flujo de datos y la dirección de control entre subrutinas:

- **Argumentos (\$a0–\$a3):** Se utilizan para transferir parámetros desde la función llamadora hacia la función llamada. Si una subrutina requiere más de cuatro parámetros, los adicionales deben pasarse a través de la pila.

- **Valores de Retorno (\$v0–\$v1):** Se emplean para devolver los resultados calculados por la subrutina hacia el llamador. Además, en entornos de simulación (como MARS o SPIM), \$v0 se utiliza para especificar el código de servicio en las llamadas al sistema (syscall).
- **Dirección de Retorno (\$ra):** Es el registro de control de flujo. Cuando se ejecuta la instrucción jal, el hardware almacena automáticamente en \$ra la dirección de la siguiente instrucción (PC+4). Al finalizar la subrutina, la instrucción jr \$ra lee este registro para regresar al punto exacto desde donde se invocó.

3. Impacto del Uso de Registros frente a Memoria en el Rendimiento de los Algoritmos de Ordenamiento

El acceso a registros del procesador se completa en un solo ciclo de reloj sin latencia, mientras que el acceso a memoria RAM mediante operaciones de carga y almacenamiento (lw y sw) introduce retardos significativos y posibles paros en la tubería (pipeline stalls).

- **Aprovechamiento de Registros:** En algoritmos como QuickSort, el recorrido y las comparaciones dentro de la rutina partition se realizan exclusivamente utilizando registros temporales (\$t0–\$t9). Al mantener los punteros e índices en registros, se minimizan las transferencias por el bus de datos.
- **Penalización por Acceso a Memoria (Stack Frame):** Cuando un algoritmo agota los registros disponibles (sobre todo en rutinas recursivas profundas o estructuras pesadas como MergeSort), se produce un register spilling. Esto obliga a escribir constantes instrucciones sw y lw hacia la pila en la memoria RAM, lo que degrada el rendimiento del algoritmo.

4. Impacto del Uso de Estructuras de Control en la Eficiencia de Algoritmos en MIPS32

El uso de estructuras de control (tales como saltos condicionales, incondicionales y bucles anidados) tiene un impacto directo y crítico en la eficiencia de los algoritmos implementados en MIPS32.

4.1 Riesgos de Control (Control Hazards) y la Segmentación del Procesador

MIPS32 implementa una tubería de procesamiento (pipeline) clásica de 5 etapas: IF (Instruction Fetch), ID (Instruction Decode), EX (Execute), MEM (Memory Access) y WB (Write Back).

Cuando el procesador ejecuta una instrucción de salto condicional (beq, bne), la dirección de destino o la evaluación de la condición no se conocen de forma inmediata en la etapa IF.

Penalización por salto (Branch Penalty): Si la tubería carga especulativamente las siguientes instrucciones continuas en memoria y la condición del salto resulta ser verdadera, esas instrucciones precargadas deben ser descartadas (flushing), lo que destruye el

trabajo adelantado y desperdicia ciclos de reloj.

Ranura de retardo de salto (Branch Delay Slot): Para reducir la pérdida de ciclos, el diseño de MIPS32 ejecuta siempre la instrucción ubicada inmediatamente después de un salto (beq, bne, j, jal). Si no se logra reorganizar el código para colocar una instrucción útil en ese espacio, es necesario insertar una instrucción nula (nop), incrementando el número total de instrucciones (I).

4.2 Costo Computacional de los Bucles Anidados

En algoritmos que requieren estructuras anidadas (como multiplicación de matrices o algoritmos de ordenamiento):

Sobrecarga de instrucciones de control: Si un bucle externo realiza N iteraciones y el interno M iteraciones, las instrucciones de salto se ejecutan un total de $O(N \times M)$ veces.

Agotamiento de registros (Register Spilling): Los bucles anidados exigen mantener múltiples contadores y punteros activos simultáneamente, forzando un intercambio constante con la pila mediante sw y lw.

4.3 Estrategias de Optimización a Nivel de Ensamblador

Desenrollado de bucles (Loop Unrolling): Duplica el cuerpo del bucle para procesar múltiples datos en una sola iteración, reduciendo las evaluaciones de saltos.

Llenado eficiente del Branch Delay Slot: Reorganizar instrucciones útiles e independientes del código para colocarlas justo después de la instrucción de salto.

Inversión de bucles (Loop Inversion): Transformar bucles while en bucles do-while precedidos por una verificación inicial para eliminar saltos incondicionales internos.

5. Diferencias de Complejidad Computacional entre Quicksort y Mergesort e Implicaciones en MIPS32

5.1 Cuadro Comparativo de Complejidad Computacional

Criterio	Quicksort	Mergesort
Complejidad Temporal (Mejor Caso)	$O(n \log n)$	$O(n \log n)$
Complejidad Temporal (Promedio)	$O(n \log n)$	$O(n \log n)$
Complejidad Temporal (Peor Caso)	$O(n^2)$ (pivote desfavorable)	$O(n \log n)$ (garantizado)
Complejidad Espacial Auxiliar	$O(\log n)$ (pila de llamadas)	$O(n)$ (arreglo aux.) + $O(\log n)$ (pila)
Tipo de Algoritmo	In-place (In situ)	Out-of-place (Requiere espacio extra)
Estabilidad	Inestable	Estable

5.2 Implicaciones para la Implementación en MIPS32

Gestión de Memoria y Pila (Stack): Quicksort solo requiere manipular el puntero de pila \$sp para guardar registros e índices en cada nivel recursivo. Mergesort exige gestionar memoria adicional en la sección .data o solicitarla dinámicamente mediante syscall 9 (sbrk).

Localidad de Referencia y Caché: Quicksort presenta una excelente localidad espacial y temporal al trabajar in-place. Mergesort duplica el tráfico de datos en el bus de memoria ($2n$ lecturas + $2n$ escrituras por nivel de mezcla), causando una mayor contaminación de caché.

6. Fases del Ciclo de Ejecución de Instrucciones en la Arquitectura MIPS32 (Camino de Datos)

El procesamiento de una instrucción en MIPS32 dentro de un diseño segmentado o monociclo se divide en 5 fases consecutivas coordinadas por la Unidad de Control Principal.

Fase	Nombre	Componentes Principales	Función Principal
1. IF	Instruction Fetch	PC, Memoria Instrucciones, Sumador +4	Busca la instrucción y actualiza el PC
2. ID	Instruction Decode	Banco Registros, Extensor Signo	Decodifica, lee registros y extiende inmediato
3. EX	Execute	ALU, MUX, Control ALU	Operaciones ALU o cálculo de direcciones
4. MEM	Memory Access	Memoria de Datos (RAM)	Lectura/Escritura en memoria RAM (lw/sw)
5. WB	Write Back	Banco Registros, MUX WB	Escribe resultado de vuelta en registro destino

6.1 Descripción de las Fases

1. IF (Instruction Fetch): Se extrae la instrucción de 32 bits desde la memoria utilizando el PC, el cual se incrementa ($PC \leftarrow PC+4$).

2. ID (Instruction Decode): Se interpreta el Opcode, se leen los registros fuente (rs, rt) y el extensor convierte los 16 bits inmediatos a 32 bits.

3. EX (Execute): La ALU ejecuta la operación aritmética/lógica, calcula la dirección de memoria o evalúa la condición de salto.

4. MEM (Memory Access): Se realiza lectura o escritura en la RAM (lw/sw). Las demás instrucciones pasan directo.

5. WB (Write Back): Escribe el resultado obtenido en el registro destino del Banco de Registros.

7. Tipo de Instrucciones Usadas Predominantemente en la Práctica (R, I, J) y Por Qué

En la práctica de programación en ensamblador MIPS32 existe un marcado predominio de las instrucciones **Tipo-I**, seguidas por las **Tipo-R**, mientras que las **Tipo-J** representan un porcentaje minoritario.

Las razones principales de este predominio son:

- **Acceso a RAM y Pila:** Tanto el manejo de la pila en quicksort como la lectura/escritura de elementos en el arreglo dentro de partition dependen de instrucciones de carga y almacenamiento (lw y sw), las cuales son Tipo-I.
- **Control de Bucles y Condicionales:** La lógica de salto para iterar o salir de los ciclos (beq, bne) pertenece al formato Tipo-I.
- **Manejo de Constantes:** El ajuste del puntero de pila (addi \$sp, \$sp, -12) y el incremento de los índices del bucle (addi \$t2, \$t2, 1) requieren un operando inmediato.

7.1 Análisis Detallado por Formato

Tipo-I (Immediate / Carga y Almacenamiento / Saltos Condicionales): Representa un 50 % – 60 % del código. Cubre la interacción con la RAM, pila, constantes y control de flujo condicional.

Tipo-R (Register): Representa un 30 % – 40 %. Núcleo de cómputo de la ALU para operaciones entre registros, desplazamientos de bits (sll) y retornos (jr \$ra).

Tipo-J (Jump): Representa < 10 %. Reservado exclusivamente para saltos incondicionales directos con direcciones de 26 bits (jal y j). Ocurren con menor frecuencia que las iteraciones de bucle.

8. ¿Cómo se ve afectado el rendimiento si se abusa del uso de instrucciones de salto (j, beq, bne) en lugar de usar estructuras lineales?

El abuso de instrucciones de salto rompe dramáticamente con la ejecución secuencial, lo cual castiga severamente la segmentación (pipeline) del procesador. En la arquitectura MIPS, cada vez que se evalúa una condición con beq o bne, el procesador intenta predecir el camino para mantener el flujo constante de trabajo. Si la predicción falla (lo cual es muy común en algoritmos de ordenamiento debido a la naturaleza aleatoria de los datos), el hardware se ve obligado a vaciar todas las instrucciones que ya había pre cargado y buscar las nuevas desde la memoria. Este proceso desperdicia ciclos de reloj irrecuperables, haciendo que un código lleno de saltos innecesarios sea notablemente más lento que un bloque de instrucciones estructurado de forma lineal y predecible.

9. ¿Qué ventajas ofrece el modelo RISC de MIPS en la implementación de algoritmos básicos como los de ordenamiento?

El modelo RISC (Reduced Instruction Set Computer) ofrece una ventaja fundamental a través de su arquitectura de carga y almacenamiento (Load/Store) y su simplicidad de decodificación. Al tener instrucciones de un tamaño fijo y estándar (32 bits), el procesador no pierde tiempo averiguando qué tipo de orden está leyendo. Además, al disponer de un amplio banco de registros de propósito general (como los \$t0-\$t9 y \$s0-\$s7), MIPS permite que las variables temporales de los algoritmos de ordenamiento (como los índices, iteradores y pivotes) vivan directamente dentro del procesador. Esto minimiza enormemente la dependencia hacia la memoria RAM externa, permitiendo que la ALU resuelva la matemática del algoritmo a máxima velocidad.

10. ¿Cómo se usó el modo de ejecución paso a paso (Step, Step Into) en MARS para verificar la correcta ejecución del algoritmo?

El modo de ejecución paso a paso fue una herramienta de diagnóstico indispensable para lograr dominar la recursividad de los algoritmos planteados. Al utilizar específicamente el comando Step Into justo en las instrucciones jal (Jump and Link), fue posible auditar en cámara lenta el descenso de la ejecución. Esto permitió verificar visualmente en cada ciclo cómo el puntero de pila (\$sp) se actualizaba para reservar memoria, y comprobar que los datos vitales, como la dirección de retorno (\$ra) y los límites del arreglo, se estuvieran resguardando y restaurando en el orden exacto. Sin esta herramienta, rastrear el origen de un bucle infinito o un colapso por desbordamiento de pila habría sido una tarea titánica.

11. ¿Qué herramienta de MARS fue más útil para observar el contenido de los registros y detectar errores lógicos?

Aunque el panel lateral de registros es vital para seguir el hilo matemático de las variables locales, la ventana inferior del Data Segment (Segmento de Datos) resultó ser la herramienta definitiva para cazar errores lógicos complejos. En algoritmos de ordenamiento, el verdadero objetivo es mutar la memoria RAM. Observar el bloque de direcciones base (por ejemplo, a partir de 0x10010000) en tiempo real permitió ver exactamente cómo y cuándo los valores del arreglo intercambiaban sus posiciones tras cada instrucción sw. Esto hizo muy evidente cuándo los cálculos de los desplazamientos (offsets) estaban fallando, ya que se reflejaba visualmente si los datos se estaban sobrescribiendo fuera de los límites del arreglo original o si el arreglo temporal no se estaba llenando correctamente.

12. ¿Cómo puede visualizarse en MARS el camino de datos para una instrucción tipo R?

Utilizando la herramienta integrada Datapath Viewer de MARS, se comprueba visualmente que una instrucción tipo R es una operación estrictamente confinada al procesador, diseñada para máxima velocidad. Al ejecutar un add, el diagrama ilumina el recorrido de la instrucción saliendo del PC, decodificando los dos registros fuente en el Banco de Registros y entrando directamente a la ALU. El aspecto más crucial de esta visualización es el final del recorrido: tras ejecutar la suma, el cable de datos esquivando por completo el bloque físico de la Memoria de Datos (Data Memory), enviando el resultado directo por el multiplexor de regreso hacia los registros. Esto confirma que la instrucción nunca interactúa con la RAM principal.

13. ¿Cómo puede visualizarse en MARS el camino de datos para una instrucción tipo I?

A diferencia del camino rápido de la tipo R, la visualización de una instrucción tipo I como el lw en el Datapath Viewer muestra el ciclo de ejecución más extenso y costoso del diagrama. El simulador ilustra cómo el valor de la dirección base sale de los registros y se dirige a la ALU. Sin embargo, aquí la ALU no realiza operaciones entre variables, sino que suma el registro base con la extensión de signo del desplazamiento (offset) para calcular la dirección física exacta. A partir de ahí, la señal luminosa entra obligatoriamente en el bloque de Memoria de Datos, donde el sistema sufre un retardo físico extrayendo la información, para finalmente encender el cable de regreso (Write Back) hacia el Banco de Registros.

14. Justificar la elección del algoritmo alternativo

Se seleccionó Mergesort como algoritmo alternativo para generar un contraste directo y medible en cuanto a la manipulación de memoria a bajo nivel. Mientras que Quicksort se caracteriza por ser inmensamente eficiente en memoria porque ordena los elementos “in-place” (sobre las direcciones originales de su propio arreglo), Mergesort impone la necesidad obligatoria de crear estructuras secundarias. Implementar esto en ensamblador sirvió como un excelente reto arquitectónico, ya que obligó a manejar dos punteros de memoria distintos en paralelo y a realizar copias masivas de datos entre la RAM y los registros durante su fase de mezcla, lo que refleja un escenario mucho más pesado para el sistema físico.

15. Análisis y Discusión de los Resultados

La conclusión principal extraída de esta práctica es que el análisis teórico de complejidad matemática ($O(n \log n)$) no refleja la totalidad del panorama cuando se enfrenta a las limitaciones físicas del hardware. Teóricamente, Mergesort garantiza ese tiempo de ejecución sin importar el desorden de los datos. Sin embargo, en la práctica dentro del entorno MIPS32, la implementación de Quicksort demostró un rendimiento superior y

mucho más fluido. La necesidad arquitectónica de Mergesort de leer, guardar y trasladar datos constantemente hacia un arreglo auxiliar generó un tráfico de instrucciones lw y sw asfixiante. Esto ratifica un principio fundamental del diseño a bajo nivel: la memoria RAM es el principal cuello de botella del computador; un algoritmo que maximiza el uso de los registros internos y minimiza el acceso a memoria siempre dominará en la ejecución física.